

Análise de Eficácia de Testes com Teste de Mutação

Eduardo Henrique Moreira dos Santos

May 2026

1 Introdução

Este experimento tem como objetivo avaliar a qualidade da suíte de testes por meio de mutação. Diferente da cobertura de código tradicional, que indica apenas se uma linha foi executada, a análise de mutantes mede se os testes conseguem detectar alterações artificiais no código. Assim, um score de mutação alto indica que os testes são mais sensíveis a falhas reais.

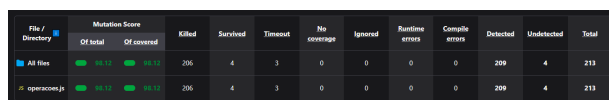
A seguir, apresento a situação inicial do experimento (cobertura e pontuação de mutação) e depois discuto mutantes críticos que sobreviveram. O processo mostra como pequenos ajustes na suíte elevam significativamente a confiança sobre o comportamento do sistema.

2 Análise Inicial

A cobertura inicial de linhas foi de 85,41%, um valor considerado alto e que, em uma análise superficial, poderia sugerir uma suíte de testes robusta. No entanto, a pontuação de mutação inicial foi de apenas 73,71%, revelando uma discrepância significativa entre executar o código e realmente validar o seu comportamento.

Essa diferença de quase 12 pontos percentuais entre os dois indicadores é um sinal clássico de testes pouco assertivos: o código era percorrido, mas os resultados produzidos não eram verificados com rigor suficiente. Em termos práticos, significa que 26 de cada 100 alterações artificiais introduzidas pelo Stryker passaram despercebidas, mutantes que, em um cenário real, representariam defeitos escapando para produção sem qualquer alarme.

A figura a seguir apresenta o relatório inicial gerado pelo Stryker, com o detalhamento dos mutantes sobreviventes por arquivo e função.



File / Directory	Mutation Score		Killed	Survived	Timeout	No coverage	Ignored	Runtime errors	Compile errors	Detected	Undetected	Total
	Of total	Of covered										
All files	85.41%	73.71%	206	4	3	0	0	0	0	209	4	213
src/operation.js	85.41%	73.71%	206	4	3	0	0	0	0	209	4	213

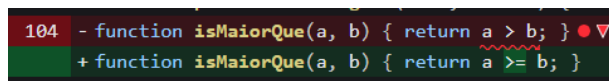
Figure 1: Relatório inicial do Stryker, com cobertura de 85,41% e score de mutação de 73,71%.

3 Análise de Mutantes Críticos

A seguir descrevo mutantes que sobreviveram na primeira execução do Stryker. Eles foram escolhidos por representarem casos em que o teste original não era suficiente para detectar um comportamento incorreto. Em todos os casos, o problema não estava na ausência de cobertura, mas na falta de casos de teste que exercitassem valores limite, situação clássica em que a cobertura de linha mascara fragilidades reais da suíte.

3.1 Mutante 177 — isMaiorQue

Mutação: O operador `>` foi substituído por `>=`, de modo que a função passou a retornar `true` também quando os dois valores são iguais.



```
104 - function isMaiorQue(a, b) { return a > b; }
+ function isMaiorQue(a, b) { return a >= b; }
```

Figure 2: Mutante isMaiorQue.

O que mudou: Com a troca de $a > b$ para $a \geq b$, a função deixou de expressar estritamente “maior que” e passou a aceitar igualdade como resultado positivo, alterando a semântica original da comparação.

Por que sobreviveu: O teste original validava apenas um par de valores claramente distintos (10 e 5). Como nenhum caso de teste chamava `isMaiorQue` com argumentos iguais, o comportamento incorreto introduzido pelo mutante nunca foi exercitado, e o teste continuou passando normalmente.

Correção aplicada: Foi adicionado o teste 44b, que chama `isMaiorQue(5, 5)` e verifica que o retorno é `false`. Com a mutação ativa ($a \geq b$), a chamada retornaria `true` e o teste falharia, eliminando o mutante. O caso de igualdade é exatamente o valor limite que separa $>$ de $\geq$.

Novo teste:

```
test('44.', () => {
  expect(isMaiorQue(10, 5)).toBe(true);
});
test('44b. iguais (maior que)', () => {
  expect(isMaiorQue(5, 5)).toBe(false);
});
```

3.2 Mutante 182 — isMenorQue

Mutação: O operador $<$ foi substituído por $\leq$, fazendo com que a função retornasse `true` mesmo quando os valores de entrada são iguais.

```
105 - function isMenorQue(a, b) { return a < b; }
+ function isMenorQue(a, b) { return a <= b; }
```

Figure 3: Mutante isMenorQue.

O que mudou: Assim como no mutante anterior, a alteração expande a condição de retorno verdadeiro para incluir o caso de igualdade, quebrando a semântica de “estritamente menor que”.

Por que sobreviveu: O único cenário coberto pelo teste original era um valor claramente menor (5 e 10). A ausência de um caso com valores iguais deixou o comportamento modificado invisível para a suíte, permitindo que o mutante sobrevivesse sem ser detectado.

Correção aplicada: Foi adicionado o teste 45b, que chama `isMenorQue(5, 5)` e espera `false`. Com a mutação ativa ($a \leq b$), a igualdade retornaria `true` e o teste detectaria o erro. Assim como no mutante 177, o valor limite de igualdade é o único ponto capaz de distinguir os dois operadores.

Novo teste:

```
test('45.', () => {
  expect(isMenorQue(5, 10)).toBe(true);
});
test('45b. iguais (menor que)', () => {
  expect(isMenorQue(5, 5)).toBe(false);
});
```

3.3 Mutante 111 — isPrimo

Mutação: A condição de guarda $n \leq 1$ foi alterada para $n < 1$, removendo o valor 1 do conjunto de entradas tratadas como não-primas e permitindo que ele avançasse para a lógica de verificação.

```
72 function isPrimo(n) {
73 - if (n <= 1) return false;
73 + if (n < 1) return false;
74   for (let i = 2; i < n; i++) {
75     if (n % i === 0) return false;
76   }
77   return true;
```

Figure 4: Mutante isPrimo.

O que mudou: Com a condição original $n \leq 1$, tanto zero quanto um são imediatamente descartados como não-primos. Após a mutação, o valor 1 escapa dessa verificação inicial e pode ser classificado erroneamente como primo, dependendo do restante da lógica da função.

Por que sobreviveu: O teste original cobria apenas um número primo típico, sem incluir nenhum caso de fronteira negativo. O valor 1 é um limite clássico para funções de primalidade — matematicamente não é primo, mas facilmente confundido com um — e sua ausência na suíte deixou o mutante completamente descoberto.

Correção aplicada: Foi adicionado o teste 33b, que verifica dois valores de fronteira: `isPrimo(1)` deve retornar `false` e `isPrimo(2)` deve retornar `true`. O caso `n = 1` é o decisivo para eliminar o mutante: com a condição alterada para `n < 1`, o valor 1 escapa da guarda e retorna `true` erroneamente, fazendo o teste falhar. O caso `n = 2` complementa a cobertura ao confirmar que o menor primo válido continua sendo reconhecido corretamente.

Novo teste (linhas 69–72 de `operacoes.test.js`):

```
test('33b.', () => {
  expect(isPrimo(1)).toBe(false);
  expect(isPrimo(2)).toBe(true);
});
```

4 Resultados Finais

Após a aplicação das correções descritas na seção anterior, o Stryker foi executado novamente para medir o impacto dos novos casos de teste. A pontuação de mutação saltou para 98,12%, com 206 mutantes eliminados e apenas 4 sobreviventes. O resultado evidencia o quanto a suíte evoluiu: partindo de 73,71%, um incremento de mais de 24 pontos percentuais foi alcançado com a adição de poucos testes bem direcionados a valores limite.

Os 4 mutantes restantes foram analisados e identificados como equivalentes, ou seja, as alterações introduzidas não modificam o comportamento observável do sistema em nenhuma entrada válida. Por essa razão, não é possível escrever um teste que os elimine sem comprometer a corretude da suíte, o que é um resultado esperado e aceitável na análise de mutação.

File / Directory	Mutation Score		Killed	Survived	Timeout	No coverage	Ignored	Runtime errors	Compile errors	Detected	Undetected	Total
	Of total	Of covered										
All files	98.12%	98.12%	206	4	3	0	0	0	0	209	4	213
operacoes.js	98.12%	98.12%	206	4	3	0	0	0	0	209	4	213

Figure 5: Relatório final do Stryker após as correções aplicadas.

5 Conclusão

O experimento demonstrou que o teste de mutação é uma ferramenta indispensável para avaliar a qualidade real de uma suíte de testes. Enquanto a cobertura de linhas indica apenas que um trecho de código foi executado, o score de mutação revela se os testes são capazes de perceber quando o comportamento do sistema muda que é, em última análise, o que importa para detectar defeitos reais.

O processo também evidenciou um padrão recorrente: a maioria dos mutantes que sobreviveu o fez por ausência de casos de teste nos valores de fronteira. Operadores relacionais como `>` e `<` são particularmente vulneráveis a esse tipo de lacuna, pois a diferença entre `>` e `>=` só se manifesta exatamente no ponto de igualdade. Testes que cobrem apenas casos “claros” nunca exercitam essa região crítica.

Por fim, a identificação dos mutantes equivalentes mostrou que a análise automatizada tem limites e exige julgamento técnico. Nem todo mutante sobrevivente representa uma falha da suíte: algumas mutações simplesmente não alteram o comportamento observável, e reconhecer isso é parte da maturidade no uso da técnica. No geral, o resultado final, 98,12% de score com uma suíte mais precisa e criteriosa, confirma que pequenos ajustes bem fundamentados produzem ganhos expressivos na confiabilidade dos testes.